Professional DevOps Study Notes

Terraform Part 1

Terraform Fundamentals

Beginner to Advanced Interview Level • English + Kannada Explanations

Learning Objectives

By the end of this part, you should be able to:

- Explain Terraform and Infrastructure as Code (IaC).
- Understand declarative and imperative approaches.
- Describe Terraform architecture and its main components.
- Explain providers, resources, data sources, and state.
- Understand the complete Terraform workflow from writing configuration to managing AWS infrastructure.
- Answer basic, scenario-based, and advanced Terraform interview questions.
- Connect Terraform concepts to real-world AWS, Jenkins CI/CD, and EKS projects.

Topic 1 — Introduction to Terraform

1. Definition — What is Terraform?

Terraform is an Infrastructure as Code (IaC) tool developed by HashiCorp. It allows you to define, provision, modify, and manage infrastructure using configuration files instead of creating every resource manually through a cloud console.

Terraform uses a configuration language called HCL (HashiCorp Configuration Language).

You can use Terraform to manage resources such as:

- AWS EC2 instances, VPCs, subnets, security groups, S3 buckets, and EKS clusters.
- Azure virtual machines and networking.
- Google Cloud virtual machines and networks.
- Kubernetes resources, depending on the provider.
- Infrastructure and services exposed through supported providers.

> ಕನ್ನಡ Terraform ಅಂದರೆ infrastructure ಅನ್ನು manual ಆಗಿ create ಮಾಡುವ ಬದಲು configuration files ಮತ್ತು code ಬಳಸಿ create, update, manage ಮಾಡುವ tool.

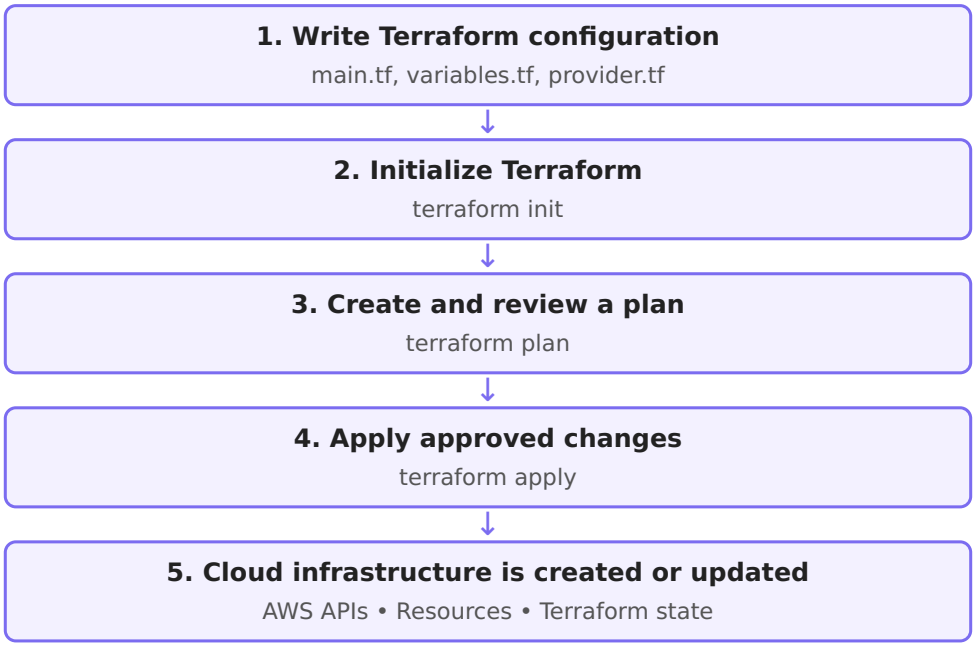
2. Purpose — Why do we use Terraform?

The main purpose of Terraform is to automate infrastructure provisioning and make it consistent, repeatable, reviewable, and easier to maintain.

Without Terraform	With Terraform
Create resources manually	Define resources in code
Repeat console operations	Reuse configuration
Difficult to reproduce environments	Recreate environments consistently
Manual changes are harder to track	Store configuration in Git
Changes can be difficult to review	Review changes with terraform plan
Infrastructure setup takes longer	Automate provisioning through CI/CD

> ಕನ್ನಡ ಒಂದೇ ರೀತಿಯ infrastructure ಅನ್ನು development, testing ಮತ್ತು production environments ನಲ್ಲಿ ಮತ್ತೆ ಮತ್ತೆ create ಮಾಡಬೇಕಾದರೆ Terraform time save ಮಾಡುತ್ತದೆ ಮತ್ತು human errors ಕಡಿಮೆ ಮಾಡಲು ಸಹಾಯ ಮಾಡುತ್ತದೆ.

3. Working — How does Terraform work?



Terraform compares the desired configuration with the state it tracks and the infrastructure information it can retrieve from the provider. It then determines which changes are needed to move toward the desired configuration.

> Important: Terraform does not automatically create resources just because you write a .tf file. You must initialize the working directory, review the plan, and apply the changes.

4. Practical Example — Create an AWS EC2 instance

This example defines an EC2 instance using an AWS provider. The AMI ID must be valid in the selected AWS region, and AWS credentials must be configured securely.

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "demo" {
  ami           = "ami-0123456789abcdef0" # Example only
  instance_type = "t3.micro"

  tags = {
    Name = "terraform-demo-instance"
  }
}
```

The AMI value above is a placeholder, not a guaranteed valid AMI. Replace it with a suitable AMI for your region. Run these commands from the directory containing the configuration:

```
terraform init
terraform fmt
terraform validate
terraform plan
terraform apply
```

To remove this demo instance later, review the impact before running:

```
terraform plan -destroy
terraform destroy
```

ಕನ್ನಡ Code ಬರೆಯುವುದರಿಂದ EC2 ತಕ್ಷಣ create ಆಗುವುದಿಲ್ಲ. init providers download ಮಾಡುತ್ತದೆ, plan changes ತೋರಿಸುತ್ತದೆ, apply AWS API ಮೂಲಕ resource create ಮಾಡುತ್ತದೆ.

5. Interview Questions & Answers

Q1. What is Terraform?

Answer: Terraform is an Infrastructure as Code tool that provisions and manages infrastructure through declarative configuration files. It supports multiple infrastructure platforms through providers.

Q2. Is Terraform a configuration management tool?

Answer: Terraform is primarily an infrastructure provisioning and lifecycle management tool. Tools such as Ansible are commonly used for configuration management, although their capabilities can overlap.

Q3. Does Terraform support only AWS?

Answer: No. Terraform supports many providers, including AWS, Azure, Google Cloud, Kubernetes, and other services.

Q4. What language does Terraform use?

Answer: Terraform primarily uses HCL, or HashiCorp Configuration Language. Terraform also supports JSON-formatted configuration.

Q5. What is the difference between terraform plan and terraform apply?

Answer: terraform plan previews proposed changes. terraform apply executes the changes, after appropriate review and approval.

Advanced Q6. Is Terraform completely idempotent?

Answer: Terraform is designed to converge infrastructure toward the declared configuration. Repeated applies with no configuration or infrastructure drift generally produce no changes, but external changes, provider behavior, computed attributes, and non-idempotent provisioners can affect outcomes.

Topic 2 — Infrastructure as Code (IaC)

1. Definition — What is IaC?

Infrastructure as Code (IaC) is the practice of defining and managing infrastructure through machine-readable configuration files rather than relying only on manual operations.

Infrastructure can include compute, networking, storage, access controls, databases, and Kubernetes resources.

> ಕನ್ನಡ Server, network, security group ಮುಂತಾದ infrastructure ಅನ್ನು console ನಲ್ಲಿ ಕೈಯಾರೆ create ಮಾಡುವ ಬದಲು code ಮೂಲಕ define ಮಾಡಿ manage ಮಾಡುವ ವಿಧಾನವೇ IaC.

2. Purpose — Why do we use IaC?

IaC helps teams:

1. Automate infrastructure creation.
2. Maintain consistent environments.
3. Version-control infrastructure in Git.
4. Review changes before deployment.
5. Rebuild infrastructure when required.
6. Reduce repetitive manual work.
7. Integrate infrastructure provisioning into CI/CD pipelines.
8. Support auditability and collaboration.

IaC does not automatically guarantee security or compliance. Teams must still configure permissions, protect secrets, review plans, and enforce appropriate policies.

3. Working — How does IaC work?

A typical workflow looks like this:

1. An engineer defines infrastructure in configuration files.
2. The code is committed to GitHub.
3. A Jenkins pipeline checks out the code.
4. Terraform initializes providers and the backend.
5. Terraform generates a plan.
6. The team reviews the plan and approves changes where required.
7. Terraform applies the approved changes.
8. The resulting infrastructure and state are monitored.

For example, a single project can define an AWS VPC, public and private subnets, security groups, an EKS cluster, and its node groups.

4. Practical Example — IaC in a Jenkins pipeline

A simple pipeline stage might run:

```
stage('Terraform Validate') {
    steps {
        bat 'terraform fmt -check -recursive'
        bat 'terraform validate'
    }
}
```

This is a Windows Jenkins agent example because bat is used. On a Linux agent, use sh. A later stage can run terraform plan. Applying infrastructure should be a separate controlled step with secure credentials and appropriate approvals.

> ಕನ್ನಡ Developer GitHub ಗೆ Terraform code push ಮಾಡುತ್ತಾರೆ. Jenkins code checkout ಮಾಡಿ validation ಮತ್ತು plan run ಮಾಡುತ್ತದೆ. Review/approval ನಂತರ apply ಮಾಡಿದರೆ AWS resources automate ಆಗಿ create ಅಥವಾ update ಆಗುತ್ತವೆ.

5. Interview Questions & Answers

Q1. What is Infrastructure as Code?

Answer: IaC is the practice of managing infrastructure using version-controlled, machine-readable definitions instead of relying exclusively on manual provisioning.

Q2. What are the benefits of IaC?

Answer: Automation, consistency, repeatability, version control, code review, faster provisioning, and better integration with CI/CD.

Q3. How does Terraform help implement IaC?

Answer: Terraform lets engineers define desired infrastructure in HCL, preview proposed changes, and apply those changes using provider APIs.

Q4. Can IaC eliminate all manual work?

Answer: No. Initial design, approvals, security reviews, incident response, and some operational tasks still require human involvement.

Advanced Q5. What is infrastructure drift?

Answer: Drift occurs when actual infrastructure differs from the configuration or state Terraform expects—for example, when an administrator changes a security group manually in AWS.

Advanced Q6. How can teams reduce drift?

Answer: Restrict unmanaged manual changes, run regular reviewed plans, monitor cloud changes, use controlled CI/CD workflows, and reconcile configuration when legitimate changes occur.

Topic 3 — Declarative vs. Imperative Approach

1. Definition — What are declarative and imperative approaches?

Declarative approach: You describe the desired final state of the infrastructure. The tool determines which operations are needed to reach that state.

Imperative approach: You specify the exact commands or sequence of operations that should be executed.

Terraform is primarily a declarative IaC tool.

> ಕನ್ನಡ
>
> - Declarative: “ನನಗೆ ಒಂದು EC2 instance ಬೇಕು.” ಅದನ್ನು ಹೇಗೆ create ಮಾಡಬೇಕು ಎಂಬುದನ್ನು Terraform ನಿರ್ಧರಿಸುತ್ತದೆ.
> - Imperative: “ಮೊದಲು ಈ command run ಮಾಡು, ನಂತರ ಈ command run ಮಾಡು, ಆಮೇಲೆ instance create ಮಾಡು.”

2. Purpose — Why is the distinction important?

Declarative configuration is useful when you want to describe the intended infrastructure and let Terraform calculate changes. Imperative commands are useful for explicit procedures, one-off operations, scripts, and tasks where execution order is central.

Declarative	Imperative
Defines the desired state	Defines the steps to execute
Tool calculates required changes	Author specifies operations
Terraform HCL is a common example	A shell script is a common example
Dependencies can be inferred from references	Order is often specified explicitly
Supports plan and review workflows	Requires careful command and error handling

Neither approach is universally better. The right choice depends on the task.

3. Working — How do they work?

Declarative example

```
resource "aws_s3_bucket" "logs" {
  bucket = "my-unique-company-logs-12345"
}
```

This declares an S3 bucket resource. The name must be globally unique among S3 general-purpose buckets. Applying this configuration creates the bucket if needed, subject to permissions and other constraints.

Imperative example

```
aws s3api create-bucket \
  --bucket my-unique-company-logs-12345 \
  --region us-east-1
```

This command directly requests bucket creation. If the bucket already exists or the name is unavailable, the command can fail; the command itself does not provide Terraform's configuration-driven reconciliation workflow.

4. Practical Example — AWS and EKS

Imagine your EKS project needs:

- One EKS cluster.
- Two worker nodes.
- A networking configuration.
- IAM roles and policies.

With Terraform, you define the desired resources and their relationships in configuration files. Terraform uses references and dependency information to determine the order of operations. You generally do not need to write a separate shell command for each AWS resource.

However, Terraform is not magic: a successful plan and apply depend on valid configuration, suitable AWS permissions, available quotas, correct networking, and supported service settings.

5. Interview Questions & Answers

Q1. Is Terraform declarative or imperative?

Answer: Terraform is primarily declarative. You define the desired infrastructure state, and Terraform calculates a set of actions to move toward that state.

Q2. What is the difference between declarative and imperative IaC?

Answer: Declarative IaC specifies what the final infrastructure should look like. Imperative automation specifies the steps used to create or modify it.

Q3. Can Terraform control resource creation order?

Answer: Yes. Terraform infers dependencies from resource references. You can use depends_on for dependencies that cannot be inferred from the configuration.

Q4. Is Terraform's execution order always the same as the order of resources in a file?

Answer: No. Terraform builds a dependency graph. Independent resources may be processed in parallel, while dependent resources are processed in the required order.

Advanced Q5. Does declarative mean Terraform never executes commands?

Answer: No. Terraform communicates with provider APIs and supports provisioners for certain use cases. However, provisioners are generally a last resort because they are less predictable than managing resources directly through providers.

Topic 4 — Terraform Architecture

1. Definition — What is Terraform architecture?

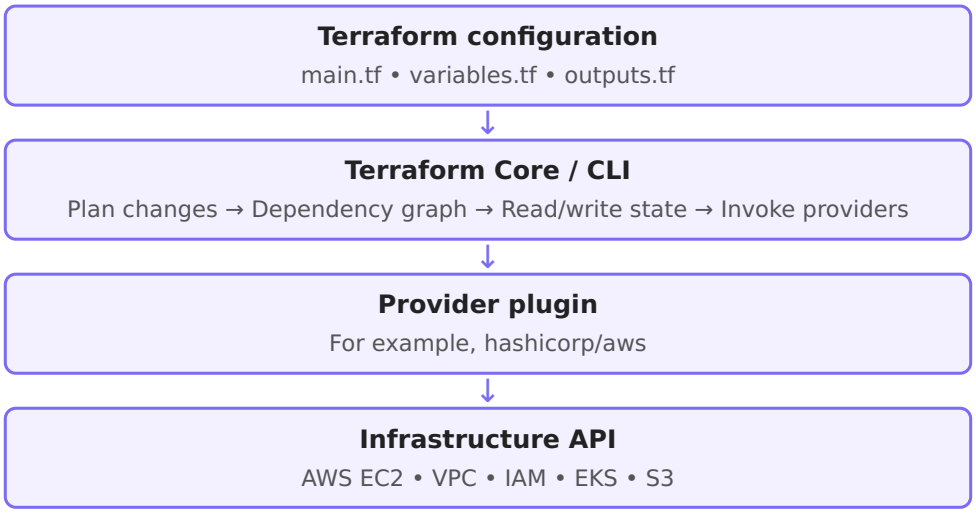
Terraform architecture describes how the Terraform CLI, configuration files, providers, state, and infrastructure work together to manage resources. The main components are:

1. Terraform CLI / Core.
2. Configuration files.
3. Providers.
4. Resources and data sources.
5. State and backend.
6. Dependency graph.
7. Infrastructure platform APIs, such as AWS APIs.

2. Purpose — Why should we understand the architecture?

- Troubleshoot provider and authentication errors.
- Understand how Terraform determines changes.
- Explain state and remote backends.
- Understand why terraform init is required.
- Diagnose dependency problems.
- Design safe Jenkins CI/CD pipelines.
- Explain Terraform in technical interviews.

3. Working — How do the components interact?



State and the configured backend support the workflow by tracking Terraform-managed objects and storing state. The state may be local or remote.

Role of each component

Component	Responsibility
Terraform CLI / Core	Loads configuration, creates a dependency graph, plans actions, and coordinates operations
Configuration	Declares the desired infrastructure

Component	Responsibility
Provider	Implements integration with a platform or service API
Resource	Represents an infrastructure object Terraform manages
Data source	Reads information about an existing object or service
State	Maps Terraform resource addresses to tracked real-world objects and stores relevant attributes
Backend	Determines where Terraform state is stored and may provide state locking
Infrastructure API	Performs the actual cloud or service operations

> Important distinction: Terraform Core does not independently create an EC2 instance. Terraform Core coordinates the operation; the AWS provider communicates with AWS APIs to manage the instance.

4. Practical Example — Terraform architecture in an AWS project

Suppose your configuration contains:

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = "t3.micro"
}
```

Here is the interaction:

1. Terraform reads the configuration.
2. The AWS provider is selected and initialized.
3. Terraform loads the available state and determines the planned changes.
4. Terraform produces a plan.
5. On apply, the AWS provider sends the relevant requests to AWS.
6. Terraform records the resulting resource information in state.

For a Jenkins job, the same architecture applies. Jenkins starts Terraform commands; Jenkins itself is not the Terraform provider and does not directly manage EC2 resources unless you explicitly automate those operations separately.

5. Interview Questions & Answers

Q1. What are the main components of Terraform architecture?

Answer: Terraform CLI/Core, configuration, providers, resources, data sources, dependency graph, state, backend, and the infrastructure APIs being managed.

Q2. What is Terraform Core responsible for?

Answer: It loads and evaluates configuration, builds the dependency graph, compares desired configuration with tracked infrastructure information, creates plans, and coordinates provider operations.

Q3. What is a provider plugin?

Answer: A provider is a plugin that allows Terraform to interact with an external platform or service. For AWS, the common provider source is hashicorp/aws.

Q4. Where is Terraform state stored?

Answer: It may be stored locally in terraform.tfstate or remotely through a configured backend such as an S3 backend.

Advanced Q5. What is Terraform's dependency graph?

Answer: It is the graph Terraform uses to determine ordering and parallelism among operations. Dependencies are commonly inferred from references between resources and can be declared explicitly with depends_on when needed.

Advanced Q6. What happens if a provider plugin cannot be downloaded?

Answer: Initialization can fail. Check network connectivity, provider source and version constraints, registry availability, proxy settings, the dependency lock file, and any configured provider mirror.

Topic 5 — Terraform Configuration Files and HCL

1. Definition — What is HCL?

HCL stands for HashiCorp Configuration Language. It is the main language used to describe Terraform infrastructure. Terraform configuration commonly uses .tf files. Files ending in .tf.json can also represent configuration in JSON syntax. Common filenames include:

File	Typical purpose
main.tf	Main resources and configuration
provider.tf	Provider configuration and requirements
variables.tf	Input variable declarations
outputs.tf	Output declarations
locals.tf	Local values and expressions
versions.tf	Terraform and provider version constraints
terraform.tfvars	Input variable values, when used

These filenames are conventions, not strict requirements. Terraform loads the relevant configuration files in the working directory together.

2. Purpose — Why use HCL?

HCL makes infrastructure configuration:

- Readable to humans.
- Suitable for version control.
- Reusable through variables and modules.
- Reviewable before deployment.
- Structured around resources and their properties.

> ಕನ್ನಡ HCL ಒಂದು readable configuration language. Infrastructure ಏನು ಬೇಕು, ಯಾವ properties ಬೇಕು ಎಂಬುದನ್ನು .tf files ನಲ್ಲಿ ಬರೆಯುತ್ತೇವೆ.

3. Working — How does HCL work?

A typical Terraform resource block has this structure:

```
resource "RESOURCE_TYPE" "LOCAL_NAME" {
  argument = value
}
```

For example:

```
resource "aws_s3_bucket" "app_logs" {
  bucket = "company-app-logs-example-12345"

  tags = {
    Environment = "Development"
    ManagedBy   = "Terraform"
  }
}
```

- resource is the block type.

- aws_s3_bucket is the resource type.
- app_logs is the local name used in Terraform references.
- bucket and tags are arguments.
- Values are assigned using =.

The S3 bucket name is an example; verify that the name is available before applying.

4. Practical Example — Organizing an AWS/EKS project

A simple EKS Terraform directory might look like this:

```
EKS-TF/
├── backend.tf
├── versions.tf
├── provider.tf
├── variables.tf
├── variables.tfvars
├── eks-cluster.tf
├── eks-node-group.tf
├── iam-role.tf
├── iam-policy.tf
└── outputs.tf
```

Typical responsibilities:

- backend.tf: remote state backend configuration.
- versions.tf: required Terraform/provider versions.
- provider.tf: AWS provider configuration.
- variables.tf: variable declarations.
- variables.tfvars: environment-specific values, if your workflow loads this file.
- eks-cluster.tf: EKS cluster resources.
- eks-node-group.tf: worker node group resources.
- iam-role.tf and iam-policy.tf: IAM resources.
- outputs.tf: useful values such as cluster identifiers or endpoints.

> Security note: Do not commit AWS secret keys, private keys, passwords, or sensitive production variable values to Git. Add appropriate ignore rules and use secure credentials management.

5. Interview Questions & Answers

Q1. What is HCL?

Answer: HCL is HashiCorp Configuration Language, used to write human-readable Terraform configuration.

Q2. Is main.tf mandatory?

Answer: No. It is a common naming convention. Terraform reads applicable configuration files from the working directory regardless of whether the resources are placed in main.tf or other .tf files.

Q3. What is the purpose of variables.tf?

Answer: It declares input variables, including their types, descriptions, defaults, and validation rules where appropriate.

Q4. What is the purpose of outputs.tf?

Answer: It declares values Terraform exposes after operations, such as an EC2 public IP or EKS cluster endpoint.

Advanced Q5. What is the difference between terraform.tfvars and variables.tf?

Answer: variables.tf normally declares the input variables. terraform.tfvars supplies values for those variables. Terraform automatically loads terraform.tfvars and certain other standard variable files; a file named variables.tfvars is not automatically loaded by that name alone. Supply it explicitly with -var-file=variables.tfvars.

Topic 6 — Terraform Providers

1. Definition — What is a provider?

A provider is a Terraform plugin that enables Terraform to manage resources and read data from an external platform, cloud service, or API. Examples include:

- AWS provider — hashicorp/aws
- AzureRM provider — hashicorp/azurerm
- Google Cloud provider — hashicorp/google
- Kubernetes provider — hashicorp/kubernetes

The provider source identifies where Terraform obtains the plugin. A provider configuration supplies settings such as region or endpoint.

2. Purpose — Why do we use providers?

Terraform needs a provider to understand how to interact with a particular platform. For example, the AWS provider enables Terraform to manage EC2 instances, security groups, S3 buckets, IAM resources, and EKS clusters through AWS APIs.

> ಕನ್ನಡ Terraform ಗೆ AWS ಅಥವಾ Kubernetes ಜೊತೆ communicate ಮಾಡಲು provider ಬೇಕು. AWS provider ಎಂದರೆ AWS services manage ಮಾಡಲು Terraform ಗೆ ಸಹಾಯ ಮಾಡುವ plugin.

3. Working — How does a provider work?

A typical provider setup has three parts:

1. Required provider declaration: identifies the provider source and version constraints.
2. Provider configuration: sets values such as region.
3. Resources and data sources: use that provider to manage or read infrastructure.

Example:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

The constraint ~> 6.0 permits compatible versions in the 6.x series below 7.0. The exact selected version is recorded in .terraform.lock.hcl after initialization.

Provider authentication

For AWS, Terraform can use supported credential sources, including:

- IAM roles attached to AWS compute.
- Environment variables.
- Shared AWS credentials/configuration files.
- Web identity or other supported role-assumption mechanisms.

For Jenkins, prefer an IAM role where the agent environment supports it, or Jenkins-managed AWS credentials with appropriately limited permissions. Avoid hardcoding access keys in Terraform files.

4. Practical Example — AWS provider in an EKS project

```
provider "aws" {
  region = var.aws_region
}

variable "aws_region" {
  description = "AWS region for the EKS infrastructure"
  type        = string
  default     = "us-east-1"
}
```

You can then define an EKS cluster resource using the AWS provider. If your provider is configured for us-east-1, the relevant AWS resources are managed in that region unless a resource or additional provider configuration specifies otherwise.

Useful commands:

```
terraform init
terraform providers
terraform version
```

terraform providers shows the provider requirements associated with the configuration and state.

5. Interview Questions & Answers

Q1. What is a Terraform provider?

Answer: A provider is a plugin that integrates Terraform with an external API or platform to read and manage resources.

Q2. What is the difference between required_providers and provider?

Answer: required_providers specifies the provider source and version constraints. The provider block configures a provider instance, for example by setting its AWS region.

Q3. What is .terraform.lock.hcl?

Answer: It records selected provider versions and package checksums to help ensure consistent provider installation. Commit it to Git for a normal project workflow.

Q4. Can one Terraform configuration use multiple providers?

Answer: Yes. A configuration can use multiple providers, including multiple configurations of the same provider using aliases.

Advanced Q5. What is a provider alias?

Answer: An alias identifies an additional provider configuration, such as an AWS provider for a second region.

```
provider "aws" {
  region = "us-east-1"
}

provider "aws" {
  alias  = "ohio"
  region = "us-east-2"
}
```

A resource can select the aliased provider using provider = aws.ohio.

Advanced Q6. What is the difference between a provider version constraint and the lock file?

Answer: The constraint defines which versions are acceptable. The lock file records the selected version and checksums. Running terraform init -upgrade can select a newer version allowed by the constraints and update the lock file.

Topic 7 — Terraform Resources

1. Definition — What is a resource?

A resource is a Terraform configuration block that declares an infrastructure object Terraform should manage. Examples:

- aws_instance — EC2 instance.
- aws_vpc — VPC.
- aws_security_group — security group.
- aws_s3_bucket — S3 bucket.
- aws_eks_cluster — EKS cluster.
- aws_eks_node_group — EKS managed node group.

2. Purpose — Why do we use resources?

Resources define the infrastructure you want Terraform to create, update, or delete. They enable you to:

- Declare desired properties.
- Express dependencies between infrastructure objects.
- Manage resource lifecycles.
- Review planned changes before applying them.
- Track managed infrastructure through Terraform state.

> ಕನ್ನಡ Resource block ಅಂದರೆ ಯಾವ infrastructure object ಬೇಕು ಮತ್ತು ಅದರ configuration ಏನು ಇರಬೇಕು ಎಂದು Terraform ಗೆ ತಿಳಿಸುವ block.

3. Working — How does a resource work?

A resource has a type and a local name:

```
resource "aws_instance" "web" {
  ami           = var.ami_id
  instance_type = "t3.micro"
}
```

The resource address is:

```
aws_instance.web
```

Terraform uses that address to refer to this resource in configuration and state. Resource lifecycle actions may include:

- Create.
- Update in place.
- Destroy and recreate.
- Delete.

The exact action depends on the resource type, provider behavior, configuration differences, and whether changed fields require replacement.

4. Practical Example — EC2 and security group

The following is a simplified EC2 example; var.ami_id must reference an appropriate AMI in the selected region.

```
resource "aws_security_group" "web" {
  name        = "terraform-web-sg"
  description = "Security group for web traffic"
  vpc_id      = var.vpc_id

  ingress {
    description = "HTTPS from approved clients"
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["203.0.113.10/32"]
  }
}

resource "aws_instance" "web" {
  ami                    = var.ami_id
  instance_type          = "t3.micro"
  vpc_security_group_ids = [aws_security_group.web.id]

  tags = {
    Name = "terraform-web-server"
  }
}
```

This demonstrates an explicit reference from the EC2 instance to the security group. Terraform infers that the security group must exist before the instance can be configured with its ID.

The example assumes var.vpc_id and var.ami_id are declared and valid. The example CIDR is reserved for documentation; replace it with the approved client range for a real deployment.

5. Interview Questions & Answers

Q1. What is a Terraform resource?

Answer: A resource represents an infrastructure object managed by Terraform through a provider.

Q2. What is a resource address?

Answer: A resource address identifies a resource in Terraform configuration and state. For example, aws_instance.web.

Q3. What happens if you change a resource argument?

Answer: Terraform evaluates the difference and proposes an update or replacement depending on the provider's behavior and the argument being changed.

Q4. What is lifecycle in Terraform?

Answer: The lifecycle meta-argument allows you to influence resource lifecycle behavior. Common settings include create_before_destroy, prevent_destroy, and ignore_changes.

Advanced Q5. What is implicit dependency?

Answer: A dependency Terraform infers from a reference. For example, aws_security_group.web.id creates a dependency on that security group.

Advanced Q6. What is explicit dependency?

Answer: A dependency declared using depends_on when Terraform cannot infer the relationship from ordinary references.

```
depends_on = [
  aws_iam_role_policy_attachment.eks_policy
]
```

Use depends_on only when necessary. Overusing it can reduce Terraform's ability to determine safe parallel operations.

Topic 8 — Data Sources

1. Definition — What is a data source?

A data source allows Terraform to read information about existing infrastructure or other information exposed by a provider, without declaring that object as a new resource to be created.

For example, you can query an existing AWS VPC and use its ID when creating an EC2 instance or EKS networking resources.

2. Purpose — Why do we use data sources?

Data sources are useful when:

- Infrastructure already exists.
- You need an AMI ID matching specific criteria.
- You need details about an existing VPC or subnet.
- You want to reuse shared infrastructure.
- You want to avoid hardcoding resource identifiers.

> ಕನ್ನಡ ಈಗಾಗಲೇ AWS ನಲ್ಲಿ ಇರುವ VPC, subnet ಅಥವಾ AMI details ಅನ್ನು Terraform ಮೂಲಕ read ಮಾಡಲು data source ಬಳಸುತ್ತೇವೆ. Data source ಸಾಮಾನ್ಯವಾಗಿ ಹೊಸ resource create ಮಾಡುವುದಿಲ್ಲ.

3. Working — How does a data source work?

A data source block starts with data:

```
data "aws_vpc" "selected" {
  id = var.vpc_id
}
```

You can then refer to its attributes:

```
vpc_id = data.aws_vpc.selected.id
```

Terraform reads the data through the provider. Depending on the data source and its inputs, the read may happen during planning or may be deferred until apply if required values are not yet known.

4. Practical Example — Reuse an existing AWS VPC

```
variable "vpc_id" {
  type        = string
  description = "ID of an existing VPC"
}

data "aws_vpc" "selected" {
  id = var.vpc_id
}

output "selected_vpc_cidr" {
  value = data.aws_vpc.selected.cidr_block
}
```

This reads the CIDR block of the specified VPC and exposes it as an output. It does not create a VPC. This pattern is useful in EKS projects that reuse an existing VPC rather than create an entirely new network.

5. Interview Questions & Answers

Q1. What is the difference between a resource and a data source?

Answer: A resource manages the lifecycle of an object. A data source reads information about an object or service.

Q2. Does a data source create a new EC2 instance?

Answer: No. An EC2 data source reads information about an existing instance. The aws_instance resource manages an EC2 instance lifecycle.

Q3. Can a data source read a VPC created by another Terraform configuration?

Answer: Yes, provided the VPC exists and the credentials have permission to read it.

Advanced Q4. When is a data source read?

Answer: Terraform reads it during planning when its required inputs are known and the read can be performed then. Some reads are deferred until apply when dependencies or values are not yet available.

Advanced Q5. Can data sources replace Terraform state?

Answer: No. Data sources help retrieve information, but they do not replace state tracking for resources managed by Terraform.

Topic 9 — Terraform State: Overview

1. Definition — What is Terraform state?

Terraform state is the data Terraform maintains to associate resources declared in configuration with the real-world objects managed by Terraform. State can include resource identifiers, attributes, dependency-related information, and other data needed to manage infrastructure over time.

By default, Terraform stores state locally in a file named:

```
terraform.tfstate
```

A team can configure a remote backend, such as an S3 backend, to store state centrally.

2. Purpose — Why do we need state?

Terraform state helps Terraform:

- Track which real-world objects correspond to resource addresses.
- Determine what exists and what needs to change.
- Maintain resource identity across operations.
- Record values returned by providers.
- Coordinate future plans and applies.
- Support collaboration through remote state and locking where available.

> ಕನ್ನಡ State file ಅನ್ನು Terraform ತನ್ನ managed resources ಬಗ್ಗೆ tracking information ಆಗಿ ಬಳಸುತ್ತದೆ. ಯಾವ resource ಯಾವ ID ಹೊಂದಿದೆ ಮತ್ತು ಮುಂದೆ ಯಾವ changes ಬೇಕು ಎಂದು ನಿರ್ಧರಿಸಲು ಇದು ಸಹಾಯ ಮಾಡುತ್ತದೆ.

3. Working — How does state work?

Suppose you define:

```
resource "aws_instance" "app" {
  ami           = var.ami_id
  instance_type = "t3.micro"
}
```

After a successful apply, state associates the Terraform address aws_instance.app with the actual AWS instance ID and relevant attributes. During a later plan, Terraform compares the configuration with state and refreshed provider information where applicable to determine the proposed changes.

Local state versus remote state

Local state	Remote state
Stored on a local machine	Stored in a shared backend
Simple for individual learning projects	Better suited to team collaboration
Can be lost if not backed up	Can use backend access controls and durability features
Collaboration requires extra coordination	Can support shared state and locking, depending on backend

Remote state is not automatically secure just because it is remote. Access control, encryption, backup, and recovery procedures still matter.

State security

> Terraform state may contain sensitive values. Do not casually publish, email, or commit it to a public repository. Restrict access and use an appropriately secured backend.

4. Practical Example — S3 remote state for an AWS project

A simplified backend configuration might look like:

```
terraform {
  backend "s3" {
    bucket       = "my-terraform-state-bucket"
    key          = "eks/terraform.tfstate"
    region       = "us-east-1"
    use_lockfile = true
  }
}
```

This example requires an existing S3 bucket and appropriate permissions. use_lockfile = true enables S3 lock-file-based state locking where supported by the installed Terraform version and backend configuration. Backend locking permissions must be configured appropriately.

> Important: Backend configuration cannot generally refer to ordinary input variables in the same way resource configuration can. Keep backend settings static or supply supported backend configuration through the appropriate initialization mechanism.

In your EKS project, the S3 backend is used to centralize state so that local and Jenkins workflows can operate on the same infrastructure state, provided they use the same backend configuration and workspace.

5. Interview Questions & Answers

Q1. What is Terraform state?

Answer: State maps Terraform-managed resource addresses to real-world objects and records information Terraform uses to manage those objects.

Q2. Can we manually edit the state file?

Answer: Manual editing is risky and generally discouraged. Prefer Terraform commands such as terraform state mv, terraform state rm, and terraform import or configuration-driven import blocks where appropriate.

Q3. What is remote state?

Answer: Remote state is state stored in a backend accessible outside the local working directory, enabling shared access and backend-supported collaboration features.

Q4. Why is state locking important?

Answer: Locking helps prevent conflicting operations from modifying shared state simultaneously. The actual locking capability depends on the backend and configuration.

Advanced Q5. What happens if Terraform state is deleted?

Answer: Terraform loses its tracking information, but the actual cloud resources usually remain. Running Terraform without restoring or reconstructing state can cause Terraform to propose creating duplicate resources or otherwise mismanage existing infrastructure. Restore a valid backup or carefully reconcile/import existing resources.

Advanced Q6. Does state contain secrets?

Answer: It can. Sensitive data may be present in state even if it is marked sensitive in configuration. Protect state access and storage accordingly.

Advanced Q7. What is the difference between state and configuration?

Answer: Configuration declares the desired infrastructure. State records Terraform's mapping and tracked information about managed infrastructure. Neither alone fully represents all the current real-world details in every situation.

Topic 10 — Core Terraform Workflow

1. Definition — What is the Terraform workflow?

The Terraform workflow is the sequence of commands and review steps used to initialize a working directory, validate configuration, plan changes, and apply those changes to infrastructure. The core commands are:

- terraform init
- terraform fmt
- terraform validate
- terraform plan
- terraform apply
- terraform destroy

2. Purpose — Why do we follow this workflow?

- Download the correct providers and initialize the backend.
- Catch formatting and configuration errors.
- Preview infrastructure changes before applying them.
- Reduce accidental resource destruction.
- Keep team operations consistent.
- Integrate infrastructure changes into Jenkins CI/CD.

> ಕನ್ನಡ Terraform commands ಅನ್ನು ಸರಿಯಾದ order ನಲ್ಲಿ run ಮಾಡಿದರೆ errors ಮುಂಚಿತವಾಗಿ ಪತ್ತೆಹಚ್ಚಬಹುದು ಮತ್ತು infrastructure changes ಅನ್ನು apply ಮಾಡುವ ಮೊದಲು review ಮಾಡಬಹುದು.

3. Working — Step-by-step commands

Step 1 — Initialize

```
terraform init
```

Initializes the working directory, downloads required providers, and configures the backend.

Step 2 — Format and validate

```
terraform fmt -recursive
terraform validate
```

Formats Terraform files and checks whether the configuration is structurally valid.

Step 3 — Review the plan

```
terraform plan
```

Shows the proposed resource changes without applying them.

Step 4 — Apply changes

```
terraform apply
```

Plans and applies changes after confirmation when using the normal interactive workflow.

Step 5 — Verify

```
terraform state list
terraform output
```

Lists tracked resource addresses and displays declared outputs, if any.

Additional commands to know

Command	Purpose
terraform version	Shows Terraform version
terraform providers	Shows provider requirements
terraform show	Displays a plan or state snapshot
terraform output	Displays declared outputs
terraform plan -out=tfplan	Saves a plan for later application
terraform apply tfplan	Applies a saved plan
terraform plan -destroy	Previews destruction
terraform destroy	Destroys managed resources, subject to the configuration and confirmation

> Important: A saved plan can contain sensitive information and should be protected. Plans can also become stale if relevant infrastructure or state changes after the plan was created.

4. Practical Example — Run a Terraform project from scratch

For your AWS or EKS project, start in the directory that contains the Terraform configuration and backend settings.

```
terraform version
terraform init
terraform fmt -recursive
terraform validate
terraform plan
```

If the plan is correct, review it before applying:

```
terraform apply
```

If your variable values are stored in a file named variables.tfvars, use:

```
terraform plan -var-file="variables.tfvars"
terraform apply -var-file="variables.tfvars"
```

If the project uses a remote backend, make sure you are initializing the correct backend and using the intended workspace before planning or applying.

Understanding the plan symbols

Symbol	Meaning
+	Create
-	Destroy
~	Update in place
-/+	Destroy and recreate
<=	Read a data source during apply in plan output

A plan may include more than one action for a resource. Always read the full plan, especially when it shows replacement or destruction.

5. Interview Questions & Answers

Q1. Why do we run terraform init?

Answer: It initializes the working directory, installs required provider plugins, and configures the backend.

Q2. What is the difference between terraform validate and terraform plan?

Answer: validate checks configuration syntax and internal consistency. plan evaluates proposed changes using configuration, state, and provider information.

Q3. Does terraform plan create resources?

Answer: It does not apply the planned infrastructure changes, although it can perform provider reads and data-source queries.

Q4. What does terraform apply do?

Answer: It executes the planned operations to create, update, or destroy resources. The normal interactive command asks for confirmation unless configured otherwise.

Advanced Q5. What is the advantage of a saved plan?

Answer: It allows teams to review a specific plan and apply that saved plan later. It supports controlled workflows, but the plan must be protected and checked for staleness.

Advanced Q6. Is terraform apply -auto-approve recommended in every pipeline?

Answer: No. It removes interactive confirmation, so it should be used only within a deliberately designed automation workflow with restricted permissions, plan review, appropriate approvals, and safeguards against destructive changes.

Topic 11 — Terraform in AWS, Jenkins, and EKS

1. Definition — How do these tools work together?

Terraform defines infrastructure, AWS provides the cloud services, Jenkins automates the pipeline, and EKS provides managed Kubernetes control-plane functionality. These tools have different responsibilities:

Tool	Responsibility
Terraform	Defines and manages infrastructure
AWS	Provides cloud infrastructure and services
Jenkins	Automates build, validation, plan, approval, and deployment workflows
Amazon EKS	Provides a managed Kubernetes control plane
Kubernetes	Runs and manages containerized workloads in a cluster

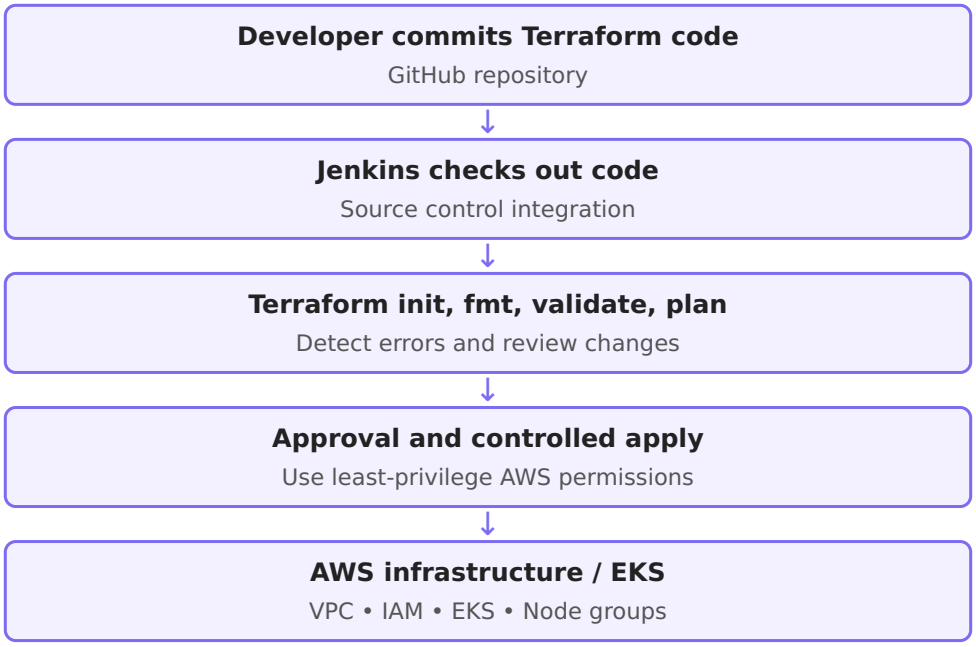
2. Purpose — Why combine them?

Combining Terraform, AWS, Jenkins, and EKS lets a team automate the infrastructure lifecycle and make changes through a consistent, reviewable workflow.

For example, Terraform can create an EKS cluster and node group. Jenkins can run Terraform checks and plans whenever the infrastructure repository changes. Kubernetes deployment automation can then deploy an application to the cluster.

> Important distinction: Provisioning the EKS cluster is not the same as deploying an application to Kubernetes. These are separate stages and often use different credentials and permissions.

3. Working — Example CI/CD flow



A separate application delivery stage may build a container image, push it to a registry such as Amazon ECR, and deploy Kubernetes manifests or Helm charts to EKS.

4. Practical Example — Jenkins pipeline for Terraform

A simplified declarative Jenkins pipeline for a Windows agent could be:

```
pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Terraform Init') {
            steps {
                bat 'terraform init'
            }
        }

        stage('Format and Validate') {
            steps {
                bat 'terraform fmt -check -recursive'
                bat 'terraform validate'
            }
        }

        stage('Terraform Plan') {
            steps {
                bat 'terraform plan -out=tfplan'
            }
        }

        stage('Approval') {
            steps {
                input message: 'Review the Terraform plan and approve apply?'
            }
        }

        stage('Terraform Apply') {
            steps {
                bat 'terraform apply tfplan'
            }
        }
    }
}
```

This is an educational skeleton, not a complete production pipeline. Before using it:

- Configure AWS credentials securely for the Jenkins agent.
- Ensure the agent has the required Terraform version.
- Initialize the correct remote backend and supply required variable files.
- Securely retain and review the exact plan produced by the pipeline.
- Add timeouts, access controls, failure handling, and approval restrictions.
- Ensure the saved plan and state are handled securely.
- Test the workflow in a non-production environment first.

In a production pipeline, restrict who can approve and apply changes, and ensure the approval corresponds to the plan that will actually be applied.

5. Interview Questions & Answers

Q1. What is Terraform's role in an EKS project?

Answer: Terraform can manage AWS networking, IAM roles and policies, EKS clusters, managed node groups, and other supporting infrastructure.

Q2. What is Jenkins' role in Terraform automation?

Answer: Jenkins orchestrates source checkout, validation, planning, approvals, and application of Terraform changes.

Q3. Why should Terraform state be remote for a team project?

Answer: A shared remote backend enables authorized team members and automation systems to use the same state. Locking, where supported and configured, helps avoid concurrent state operations.

Q4. Why should we review a Terraform plan before applying it?

Answer: It can reveal unintended resource replacements, deletions, permission changes, or unexpected infrastructure costs before those changes are applied.

Advanced Q5. Should the application deployment and infrastructure provisioning be in the same Jenkins pipeline?

Answer: They can be in the same pipeline or separate pipelines. Separate pipelines can provide clearer ownership, independent approvals, and better separation between infrastructure changes and application releases.

Advanced Q6. How should Jenkins authenticate to AWS?

Answer: Prefer short-lived credentials or an appropriately scoped IAM role where possible. Otherwise, use a supported Jenkins credentials integration, apply least privilege, and never hardcode AWS secrets in source code.

Topic 12 — Common Terraform Errors and Troubleshooting

1. Definition — What is Terraform troubleshooting?

Terraform troubleshooting is the process of identifying and fixing errors in configuration, provider installation, authentication, state management, dependency resolution, and infrastructure provisioning.

2. Purpose — Why is troubleshooting important?

In real DevOps environments, a pipeline may fail because of invalid HCL, insufficient IAM permissions, a missing variable, a provider version conflict, or a state lock. Understanding the error helps you fix the actual cause instead of repeatedly rerunning commands.

3. Working — Troubleshooting checklist

Error or situation	What to check
terraform: command not found	Terraform installation and PATH
Provider initialization failed	Network, provider source, version constraints, lock file
No valid credential sources found	AWS credential chain, IAM role, Jenkins credential configuration
AccessDenied	IAM permissions and resource policies
Missing required variable	Variable declarations and .tfvars file
State lock error	Identify the lock owner and whether another operation is active
Resource already exists	Check state and existing infrastructure before importing or changing configuration
EKS creation failed	IAM permissions, subnet configuration, quotas, version and networking requirements
Unexpected resource replacement	Review the full plan and identify which argument requires replacement

4. Practical Example — Debug an AWS authentication failure

Suppose Jenkins reports that Terraform cannot authenticate to AWS. Run the following commands in the same agent environment and under the same identity used by the Terraform pipeline:

```
aws sts get-caller-identity
terraform init
terraform plan
```

On a Windows Jenkins agent, use the corresponding bat steps. If the AWS CLI identity check fails, investigate credential configuration first. If it succeeds, verify that the identity has the required permissions for the resources and backend.

> ಕನ್ನಡ ಮೊದಲು AWS identity ಸರಿಯಾಗಿ ಬರುತ್ತಿದೆಯೇ check ಮಾಡಬೇಕು. Authentication successful ಆದರೂ AccessDenied ಬಂದರೆ, IAM permissions ಪರಿಶೀಲಿಸಬೇಕು. Authentication issue ಮತ್ತು authorization issue ಎರಡೂ ಬೇರೆ.

5. Interview Questions & Answers

Q1. What do you do when terraform init fails?

Answer: Check the complete error message, backend configuration, provider source and version, network connectivity, credentials needed for the backend, and provider mirror or registry access.

Q2. What do you do when terraform apply fails halfway through?

Answer: Read the error, verify the actual cloud resource status, and inspect state before retrying. Terraform may have completed some operations before the failure, so never assume nothing changed.

Q3. What should you do if a state lock is present?

Answer: Verify whether another operation is active. If the lock is genuinely stale, follow the backend's recovery procedure. Do not force-unlock a state that another process is actively using.

Advanced Q4. What is terraform force-unlock?

Answer: It removes a Terraform state lock after a lock identifier is supplied. It is a recovery operation and should only be used when you have verified that the lock is stale and no legitimate operation is in progress.

Advanced Q5. How do you investigate unexpected drift?

Answer: Run a reviewed plan, identify the differences, verify whether changes were made outside Terraform, and decide whether to update configuration, reconcile the infrastructure, or restore the intended state. Avoid blindly applying changes when the plan contains unexpected replacements or deletions.

Topic 13 — Scenario-Based Interview Preparation

These scenarios are particularly useful for AWS, Jenkins, and EKS DevOps interviews.

Scenario 1 — EC2 was created manually in AWS

Question: An EC2 instance already exists in AWS, but it is not tracked by your Terraform configuration. What would you do?

Answer: First verify the instance's identity, configuration, and intended ownership. Then define the matching Terraform resource and use a supported import workflow to bring the existing instance under Terraform management. Review the plan before making any changes.

> ಕನ್ನಡ ಈಗಾಗಲೇ ಇರುವ EC2 ಅನ್ನು ಮತ್ತೆ create ಮಾಡಬಾರದು. ಮೊದಲು identify ಮಾಡಿ, configuration ಬರೆಯಬೇಕು, ನಂತರ import ಮಾಡಿ plan ಪರಿಶೀಲಿಸಬೇಕು.

Scenario 2 — Jenkins plan wants to destroy an EKS cluster

Question: Your pipeline unexpectedly proposes destroying the EKS cluster. What should you do?

Answer: Stop the deployment. Review the plan, resource addresses, variable values, workspace, provider configuration, and state backend. Identify why Terraform proposes deletion and correct the cause before seeking approval to apply any changes.

> ಕನ್ನಡ Plan ನಲ್ಲಿ EKS destroy ಆಗುತ್ತದೆ ಎಂದು ತೋರಿಸಿದರೆ ತಕ್ಷಣ apply ಮಾಡಬೇಡಿ. Configuration, workspace ಮತ್ತು state ಪರಿಶೀಲಿಸಿ.

Scenario 3 — Two engineers run apply simultaneously

Question: How do you prevent concurrent operations from corrupting shared state?

Answer: Use a shared remote backend with supported state locking, ensure both engineers use the same intended backend and workspace, and coordinate deployments. Locking does not replace change review or operational coordination.

Scenario 4 — AWS credentials work locally but fail in Jenkins

Question: Why might Terraform work on your laptop but fail in the Jenkins agent?

Answer: The local user and Jenkins service may run under different operating-system accounts and credential contexts. Check the agent's identity, environment, credential binding, IAM permissions, region, and backend access.

Scenario 5 — Someone changes a security group manually

Question: How can Terraform detect and respond to the change?

Answer: A refresh-enabled plan can identify differences between the tracked configuration and actual infrastructure. Review the plan, determine whether the manual change was intentional, and reconcile the code or infrastructure appropriately.

Scenario 6 — An AWS resource already exists, but Terraform wants to create it

Question: What could cause this?

Answer: The resource may not be tracked in the selected state, or Terraform may be using a different backend or workspace. Verify the resource address, backend, workspace, and state before considering import. Do not assume the cloud resource is missing simply because it is absent from state.

Topic 14 — Quick Revision Cheat Sheet

Important concepts

Concept	Remember this
Terraform	Infrastructure provisioning and lifecycle management
IaC	Manage infrastructure through code
HCL	Terraform's main configuration language
Declarative	Define the desired state
Provider	Connects Terraform to an external platform/API
Resource	Infrastructure object Terraform manages
Data source	Reads information from an existing object or service
State	Tracks resource identity and attributes
Backend	Determines where state is stored and may support locking
Dependency graph	Determines operation dependencies and parallelism
Module	Reusable Terraform configuration
terraform init	Initialize the directory and providers/backend
terraform fmt	Format configuration
terraform validate	Validate configuration structure
terraform plan	Preview proposed changes
terraform apply	Execute planned infrastructure changes
terraform destroy	Destroy resources managed by the configuration

Essential commands to memorize

```
terraform version
terraform init
terraform fmt -recursive
terraform validate
terraform plan
terraform apply
terraform state list
terraform output
terraform plan -destroy
terraform destroy
```

Mini practical exercise

Try these questions before moving to Part 2.

1. Which Terraform command initializes providers and the backend?

A) terraform validate

B) terraform init

C) terraform output

2. Which block declares an AWS EC2 instance to be managed?

A) data "aws_instance"

B) provider "aws"

C) resource "aws_instance"

3. Which file normally records selected provider versions and checksums?

A) outputs.tf

B) .terraform.lock.hcl

C) terraform.tfvars

4. An existing VPC must be queried, not created. Which construct is appropriate?

A) A data source

B) An output block

C) A local value

5. Your EKS plan unexpectedly shows destruction. What is the safest first action?

A) Run apply immediately

B) Delete the state file

C) Stop and investigate the plan

Answer key: 1 → B (terraform init prepares the working directory and backend) • 2 → C (resource "aws_instance" manages the object lifecycle) • 3 → B (.terraform.lock.hcl) • 4 → A (a data source reads existing infrastructure) • 5 → C (understand unexpected destruction before any apply)

Final interview summary

If an interviewer asks, “Explain Terraform from scratch,” a strong answer is:

> Terraform is a declarative Infrastructure as Code tool that uses HCL to define infrastructure. It uses providers to communicate with platforms such as AWS. Terraform initializes the working directory, evaluates the configuration, uses state and provider information to calculate a dependency graph and proposed changes, and applies those changes through provider APIs. State tracks Terraform-managed resources, while a remote backend can support team collaboration and state locking. In a DevOps workflow, Jenkins can automate Terraform validation, planning, approval, and application of infrastructure changes.

> ಕನ್ನಡ Terraform ನಲ್ಲಿ infrastructure code ಮೂಲಕ define ಮಾಡುತ್ತೇವೆ. Provider AWS ಜೊತೆ communicate ಮಾಡುತ್ತದೆ. init setup ಮಾಡುತ್ತದೆ, plan changes ತೋರಿಸುತ್ತದೆ, apply changes execute ಮಾಡುತ್ತದೆ. State resources ಅನ್ನು track ಮಾಡುತ್ತದೆ; Jenkins ಈ workflow ಅನ್ನು automate ಮಾಡಬಹುದು.

Next part: Part 2 — Terraform Installation & Configuration, including installation on Windows, Terraform versions, provider setup, AWS authentication, CLI commands, backend initialization, and practical troubleshooting.